

Modern CLI Tools — Cheatsheet

Rule: new names for new tools; core commands keep core behaviour. Never alias `grep`/`find` to an incompatible tool.

rg — `grep -r`

<code>rg pattern</code>	recursive search
<code>rg -i / -w</code>	no-case / whole word
<code>rg -F 'lit.str'</code>	fixed string
<code>rg -l pat</code>	filenames only
<code>rg -tpy pat</code>	only .py (-Tpy = excl.)
<code>rg -g '*.conf' p</code>	glob filter
<code>rg -C3 pat</code>	3 lines context
<code>rg -uu pat</code>	ignore .gitignore+hidden
<code>rg --files</code>	list searchable files

ugrep (ug) = grep-flag-compatible alternative.

fd — `find`

<code>fd pattern</code>	regex on filename
<code>fd -e md</code>	by extension
<code>fd -tf / -td</code>	files / dirs
<code>fd -H / -I</code>	hidden / ignore .gitignore
<code>fd -g '*.tar.gz'</code>	glob mode
<code>fd -d 2 pat</code>	max depth
<code>fd . dir -x cmd</code>	exec per result
<code>fd -0 xargs -0</code>	null-safe pipe

fzf — interactive filter

<code>Ctrl-R</code>	fuzzy history
<code>Ctrl-T</code>	insert file path
<code>Alt-C</code>	cd into picked dir
<code>**<Tab></code>	fuzzy path completion
<code>cmd fzf -m</code>	multi-select (Tab)
<code>--preview '...{}'</code>	preview pane

In-picker: **Ctrl-J/K** move · **Tab** mark · **Ctrl-/** toggle preview

bat — `cat` (reading)

<code>bat file.py</code>	highlight + numbers
<code>bat -p file</code>	plain, copy-safe
<code>bat -A file</code>	show hidden chars
<code>bat -r 20:40 f</code>	line range
<code>bat -l json < f</code>	force language
<code>bat --diff f</code>	only git-changed lines

Pipe-safe: acts as cat when not a TTY.

eza — `ls`

<code>eza -lh --git</code>	long + git status
<code>--icons=auto</code>	Nerd Font icons
<code>eza -la</code>	long + hidden
<code>eza --tree -L2</code>	replaces tree
<code>--sort=size -r</code>	largest first
<code>--sort=modified -r</code>	newest first
<code>--total-size</code>	real dir sizes (slow)

eza is dead — use eza.

zoxide — `cd`

<code>z foo</code>	jump to best match
<code>z foo bar</code>	multi-keyword match
<code>zi foo</code>	interactive pick (fzf)
<code>z -</code>	previous dir
<code>zoxide query -l</code>	dump database

btop — `top` / `htop`

Esc menu · **f** filter · **t** tree view · **+/-** expand

1 2 3 4 toggle CPU/mem/net/proc boxes

k kill · **Enter** process detail · **q** quit

gdu — `ncdu`

<code>gdu /</code>	interactive TUI
<code>gdu -n /var</code>	non-interactive
<code>gdu -d /home</code>	show devices

In-app: **d** delete · **r** rescan · **n/s/c** sort name/size/count · **Enter** descend

duf — `df` (NOT `du`)

<code>duf</code>	all mounts
<code>duf --only local</code>	local FS only
<code>--hide-fs tmpfs</code>	hide noise
<code>duf --sort size</code>	sort

dust — `du` (non-interactive)

<code>dust</code>	size tree of cwd
<code>dust -d 2 /var</code>	limit depth
<code>dust -r</code>	reverse order

procs — `ps aux | grep`

<code>procs nginx</code>	search name/pid/user
<code>procs --tree</code>	process tree
<code>procs --sortd cpu</code>	desc by CPU
<code>procs --watch</code>	live refresh

hyperfine — `time`

<code>hyperfine 'a' 'b'</code>	compare two commands
<code>--warmup 3</code>	warmup runs
<code>-L n 1,4,8 'mk -j{n}'</code>	parameter sweep

Modern CLI Tools — Cheatsheet (2/2)

jq — JSON

```
jq . f.json
jq -r '.a[].name'
```

pretty-print
raw output, no quotes

```
jq -c '.[[]]'
'.[] | select(.n>3)'
```

one object per line
filter

```
'map({id, name})'
```

reshape array

```
jq -s 'add' a b
```

slurp & merge files

```
--arg k "$V" '.[${k}]'
```

safe var injection

```
jq 'keys'
```

object keys

```
jq 'to_entries[]'
```

k/v pairs

jqless = interactive JSON pager for exploring.

delta — git pager

```
git diff
```

via core.pager

```
delta a.py b.py
```

standalone diff

```
-c delta.side-by-side=true
```

force split view

With **navigate=true**: **n** / **N** jump between files

sd — sed s///

```
sd 'old' 'new' f
sd -p 'a' 'b' f
sd -s 'a.b' 'x' f
fd -e md -x sd a b
```

in-place replace
preview, no write
literal, no regex
bulk across files

Substitution only — sed still owns the rest.

xh — curl / httpie

```
xh url
xh POST url a=b
xh POST url n:=30
xh url q==linux
xh url X-Tok:abc
xh --download url
xh -v POST url
```

GET
JSON string field
raw JSON type
query param
header
save to file
show request too

Keep curl for scripts & images.

yazi — mc / ranger

```
h j k l
y
a
/
Enter
```

navigate · **Space** select · **v** visual select
copy · **x** cut · **p** paste · **d** delete
create · **r** rename · **.** toggle hidden
search · **F** filter · **Tab** preview
open · **q** quit+cd · **Q** quit no-cd

lazygit — git TUI

```
Space
c
b
1-5
Esc
q
```

stage/unstage · **a** stage all · **Enter** stage hunks
commit · **A** amend · **P** push · **p** pull
branches · **s** stash · **x** menu · **? help**
switch panels · **Esc** back · **q** quit

atuin — shell history

```
Ctrl-R
atuin stats
atuin import auto
```

search history
usage stats
one-time migration

Conflicts with fzf Ctrl-R — pick one.

tldr — man

```
tldr tar
tldr --update
```

practical examples
refresh cache

Category verdicts

```
exa
lsd
gojq
micro
dust ≠ ncdu
duf ≠ du
fish
oh-my-zsh
ugrep
```

dead → eza
redundant with eza
only for bigint / YAML
use it; nano = fallback
gdu is the ncdu swap
duf replaces **df**
non-POSIX → use zsh
heavy → 2 source lines
pick over rg for grep flags

Install priority

```
1. zsh + 2 plugins · rg · fzf · fd
2. zoxide · bat · eza · delta · starship
3. btop · gdu · duf · procs · atuin · tldr · lazygit
4. yazi · xh · dust · sd · micro · hyperfine
```

Additive tools (fzf, zoxide, atuin, delta, btop, hyperfine, lazygit) have no coreutils equivalent — nothing to un-learn. Insurance for future remote work: keep grep -rn, find -name -type, du -sh, ps aux.

Combos worth memorizing

```
rg -l pat | xargs sd a b
fd -tf | fzf --preview 'bat {}'
```

bulk edit every match
pick a file visually

```
git branch | fzf | xargs git switch
```

switch

```
fuzzy branch switch
curl -s $U | jq -r '.data[].id'
```

extract IDs from an API

```
rg --files | rg conf
```

find by path fragment

```
fd -e log -x gzip
```

batch-process matches

Init load order

```
0. compinit
1. fzf --bash/--zsh
2. zoxide init
3. atuin init
4. starship init
5. syntax-highlighting
```

zsh, before plugins
after cd aliases
after fzf (Ctrl-R)
absolute last (zsh)

Shared aliases in ~/.shell_common; only init lines in each rc.

zsh on WSL

```
apt install zsh
chsh -s $(which zsh)
wsl --shutdown
wsl.exe ~ -e zsh
```

install
default shell
PowerShell, to apply
Terminal profile fallback

Skip oh-my-zsh: 2 source lines (autosuggestions + syntax-highlighting) replace it.

WSL gotchas

```
/mnt/c
Nerd Font
Slow tab-completion →
appendWindowsPath=false
wsl.conf
wsl --shutdown
```

/mnt/c is slow — keep repos in ~/, never scan it with rg/fd/gdu
Nerd Font is set in the **Windows Terminal** profile, not in WSL
Slow tab-completion →
appendWindowsPath=false in /etc/wsl.conf
Any /etc/wsl.conf change needs **wsl --shutdown**